

Data Science for Economists

Lecture 10: Model Tunning and Testing

Jiaxi Li

University of North Carolina | ECON 370

Table of contents

1. Introduction
2. Simulation
3. Descriptive Statistics and Tests: LM
4. Training/Testing: LM
5. Training/Validating/Testing: Ridge
6. Bias Variance Trade-off

Introduction

Motivation

Back in Lecture 9, we fit several models, but a few puzzles still linger in the air:

1. **How do we judge model quality?** Can we quantify how well a model fits the data, and can we say with confidence that a particular variable truly relates to the outcome?
2. **How do we choose *hyperparameters*?** Machine learning methods like *Ridge* and *LASSO* introduce *tuning parameters* (sometimes called *hyperparameters*). Different datasets may prefer different settings (as you'll experience in HW4), so how do we pick these hyperparameters in a principled way?
3. **Simple model or complex model?** When is a flexible model justified, and when should we stick to something more restrained? How do we compare models in a fair, data-driven manner?

Motivation

To build intuition for the statistical tools that help answer these questions, we'll first introduce two core simulation frameworks:

- **Monte Carlo Simulation:** Used to test theories or methods. Parametric, because we assume a specific model or distribution that generates the data.
- **Bootstrap:** Used to test properties of the data itself. Nonparametric, because it relies entirely on the observed sample without imposing extra distributional assumptions.

Simulation

Review of Statistics

Here are some basic statistics theory that you might need to understand what is going on in this class even before we start doing simulation:

- All probabilities must be in $[0, 1]$ and the sum of all possible outcomes equals 1 (100%).
- Random variables (RVs) map random outcomes to real numbers
- RVs are notated with capital letters (e.g. X) whereas "realized" (i.e. nonrandom) outcomes are lowercase (e.g. x)
- The distribution of an RV has a probability function $f_X(x)$
 - If X is discrete, $f_X(x)$ is known as a probability mass function (pmf).
 - If X is continuous, $f_X(x)$ is known as a probability density function (pdf).
- The distribution of an RV also has a Cumulative Distribution Function (CDF) $F_X(x)$
 - $F_X(x) = \Pr(X \leq x)$
- The quantile function of a random variable gives $q(\alpha)$, the value such that a random draw from the distribution falls at or below $q(\alpha)$ with probability α .
- If X is continuous, $\frac{d}{dx}F_X(x) = f_X(x)$ and $F_X(x) = \int_{-\infty}^x f_X(t)dt$

"Moments"

Usually, we use "moments" to describe the characteristics of a random variable. Like the first moment of $\mathbf{X}$ is called mean/average and the second (centered) moment of $\mathbf{X}$ is called variance:

- The "average value" of an RV is known as it's expectation:
 - Discrete: $E[\mathbf{X}] = \sum_i x_i f_X(x_i)$
 - Continuous: $E[\mathbf{X}] = \int_{-\infty}^{\infty} x f_X(x) dx$

The average/expectation measures the center of data.

- The variance of a distribution is: $\text{Var}(\mathbf{X}) = E[(\mathbf{X} - E[\mathbf{X}])^2] = E[\mathbf{X}^2] - E[\mathbf{X}]^2$

The variance measures the spread of data.

- For two RVs $\mathbf{X}$ and $\mathbf{Y}$, the covariance is:

$$\text{Cov}(\mathbf{X}, \mathbf{Y}) = E[(\mathbf{X} - E[\mathbf{X}])(\mathbf{Y} - E[\mathbf{Y}])] = E[\mathbf{XY}] - E[\mathbf{X}]E[\mathbf{Y}]$$

The covariance measures the **linear** association (co-movement) between $\mathbf{X}$ and $\mathbf{Y}$.

Counterexample: circle.

Scaling/Standardizing

For both comparison and model fitting, it is common to scale or standardize variables. Given a random variable X with mean μ and variance σ^2 , its standardized form is:

$$\tilde{X} = \frac{X - \mu}{\sigma}.$$

We will see this idea appear in several places: t-statistics use standardized coefficients, and metrics such as R^2 follow the same spirit of putting quantities on a comparable scale.

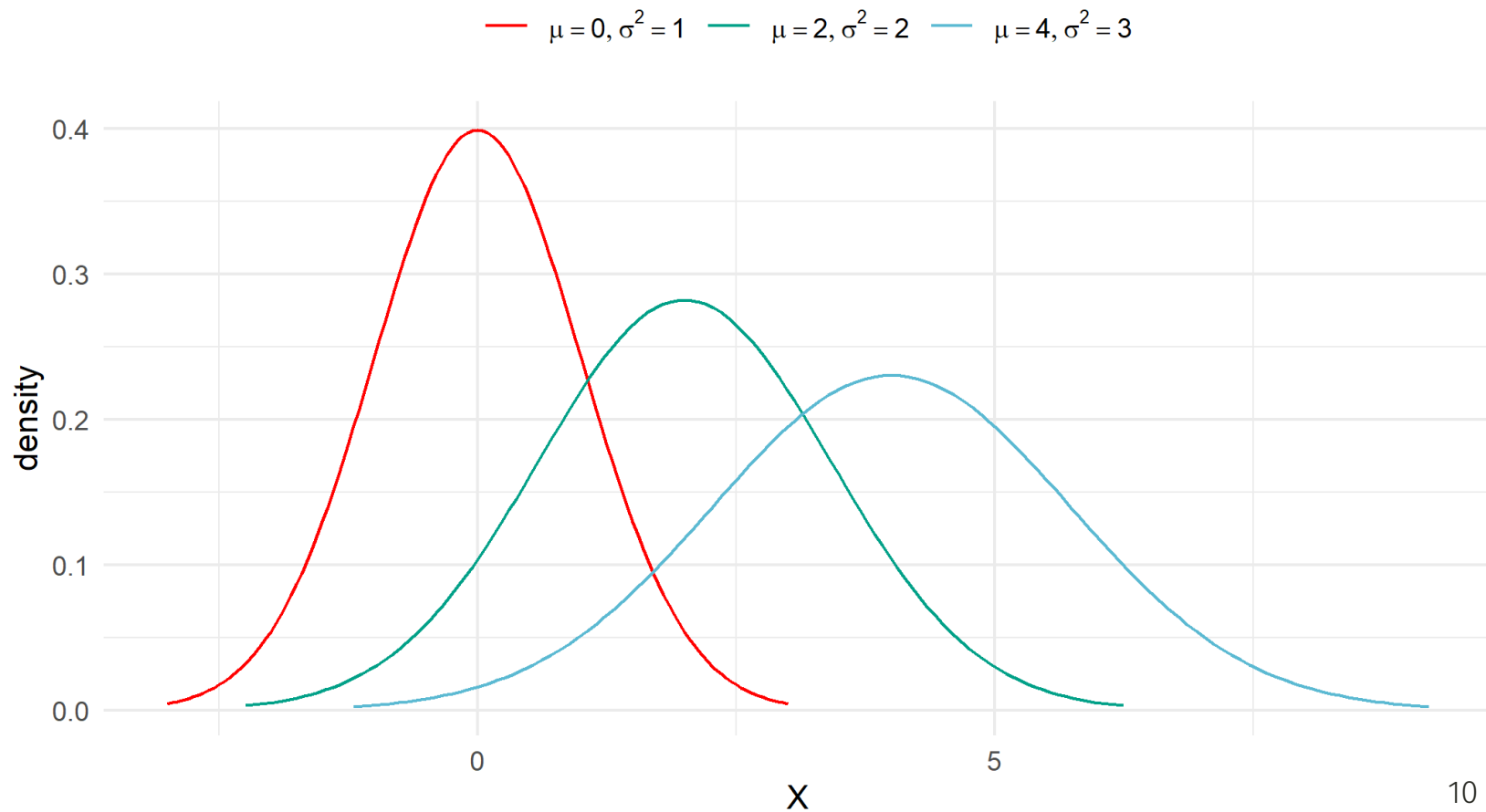
For covariance, the standardized version is the correlation, defined as:

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}}.$$

Common Distributions: Normal

The Normal Distribution parameters: μ and $\sigma^2 \geq 0$; notated $N(\mu, \sigma^2)$

- Mean and variance: μ and σ^2



Simulation

While it sounds fancy, simulation is nothing but running "data generating processes" in `R` with randomly generated data.

- What is a data generating process (DGP)?

"...a data generating process is a process in the real world that "generates" the data one is interested in."

Think of a DGP as the "real world model" that produces the data we observe.

We can never observe the true DGP.

Mathematical modeling aims to take what we observe in the real world and create a model that both replicates that data and makes useful predictions in new situations.

We can hypothesize one possible DGP and simulate the DGP to produce "fake" data.

Monte Carlo Simulation

- Sometimes we would like to examine if a statistical estimation procedure we come up with actually works the way we hope it should.
- To do this, we can simulate the DGP, perform the estimation procedure multiple times, and see if it's right "on average."
- This is called **Monte Carlo simulation**.
- We will generally apply **loops** we learned in Lecture 5 for simulations.

Review of LLN

- The Law of Large Numbers (LLN) states that if we have i.i.d. data drawn from "well behaved" distributions, as the sample size gets bigger, the population mean converges to the actual mean.
- In math,

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n X_i = E[X]$$

(Sorta, this is actually not exactly correct; the limit should actually be a probability limit, but this is the right intuition)

- Note: we've actually already used this result when using simulations to calculate expectations!
- Let's see an example.

LLN Example

- Suppose we have 1,000 draws from $U(0, 1)$.
- This is uniform distribution with theoretical mean at 0.5.
- Take cumulative sum to simulate getting an extra draw each time.

```
set.seed(2025) # Set seed for reproducibility

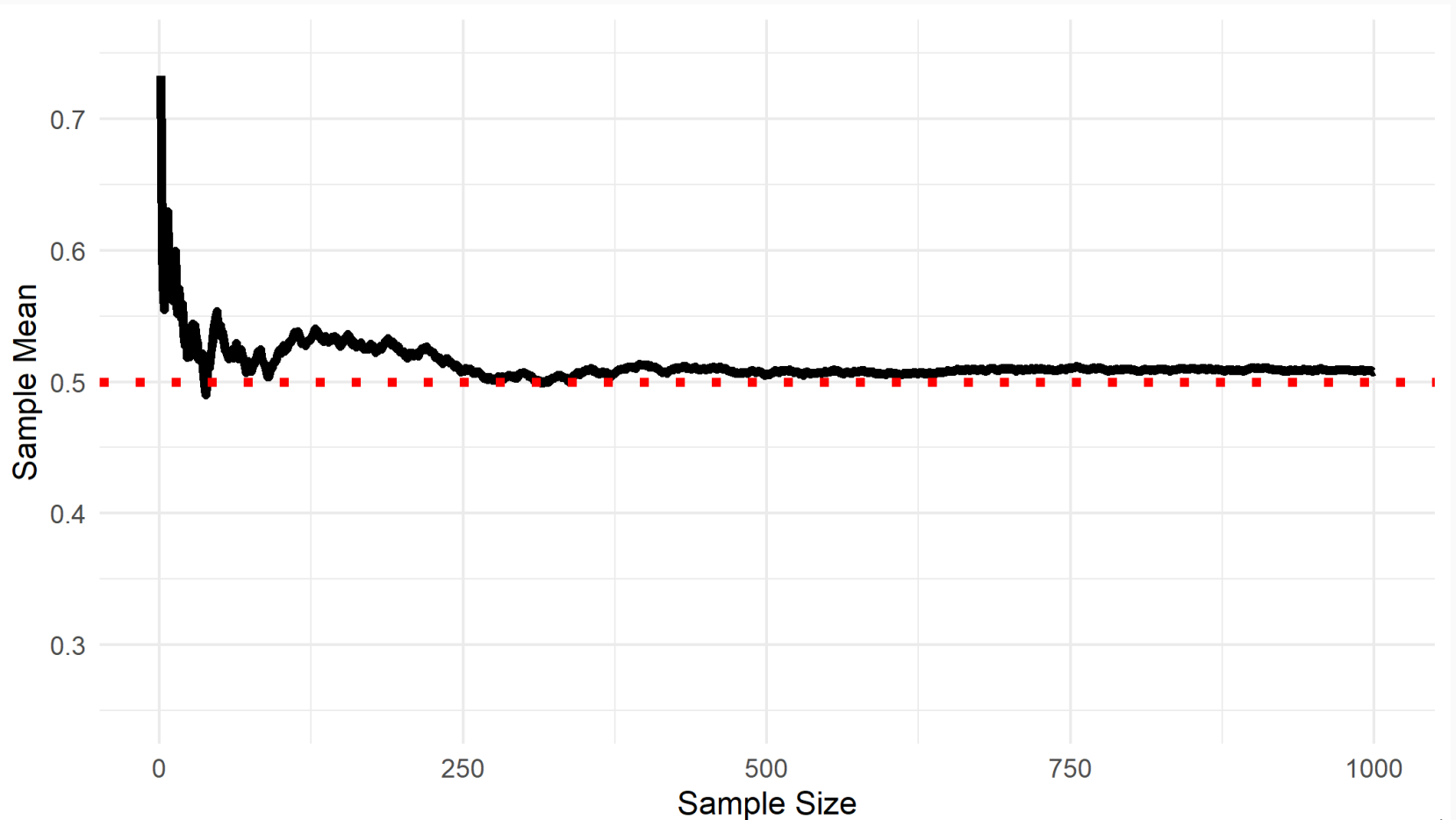
Nsim          = 1000                # Number of simulations / sample size
samp_means    = cumsum(runif(Nsim))/(1:Nsim) # Compute cumulative average of uniform(0,.

# Create data table for plotting
plot_data = data.table(x=1:Nsim, y=samp_means)

# Plot the Law of Large Numbers (LLN) demonstration for Uniform distribution
LLN_plot = ggplot(plot_data, aes(x=x, y=y)) +
  geom_line(size=1.5) + coord_cartesian(ylim=c(0.25, 0.75)) +
  geom_hline(yintercept=1/2, linetype='dotted', col = 'red', size=1.5) +
  ylab("Sample Mean") + xlab("Sample Size")+theme_minimal()
```

LLN Example

- Notice how the line gets closer to the true mean, $1/2$.
- However, improvement is not monotonic; also, after a bit, improvement is slow.



LLN Example 2

- Suppose we have 1,000 draws from **Exp**(1/2).
- This is exponential distribution (look up if you are curious) with theoretical mean at 2.
- Take cumulative sum to simulate getting an extra draw each time.

```
set.seed(2025) # Set seed for reproducibility

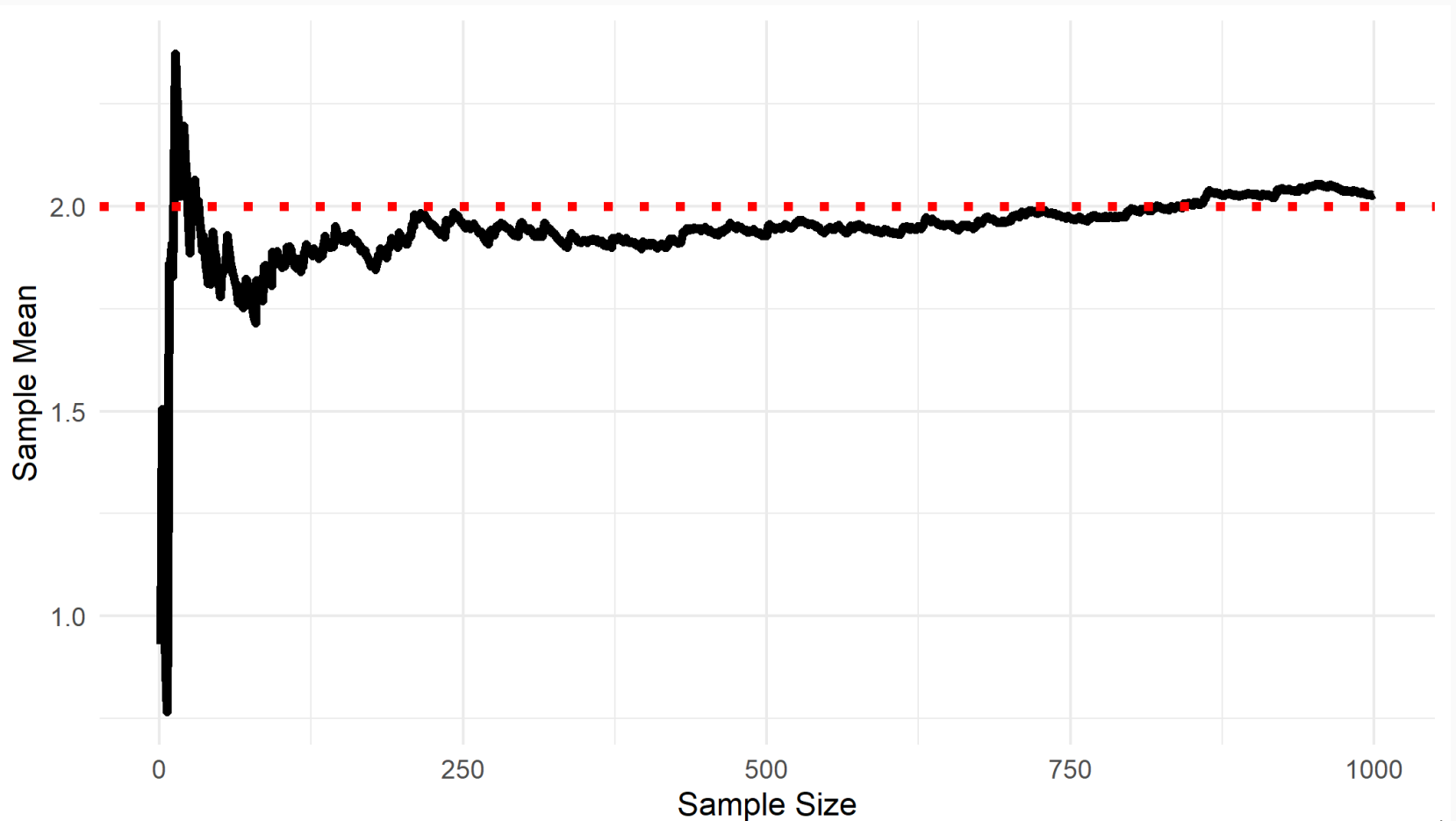
Nsim      = 1000          # Number of simulations / sample size
lambda    = 1/2           # Rate parameter for Exponential distribution
samp_means = cumsum(rexp(Nsim, lambda)) / (1:Nsim) # Cumulative average of exponential

# Create data table for plotting
plot_data = data.table(x = 1:Nsim, y = samp_means)

# Plot the Law of Large Numbers (LLN) demonstration for Exponential distribution
LLN_plot2 = ggplot(plot_data, aes(x=x, y=y)) +
  geom_line(size=1.5) + ylab("Sample Mean") + xlab("Sample Size") +
  geom_hline(yintercept=2, linetype='dotted', col = 'red', size=1.5) +
  theme_minimal()
```


LLN Example 2

- Notice how the line gets closer to the true mean, 2.
- Notice that this one appears to converge faster than the uniform example.



LLN Example 3

- Suppose we have 1,000 draws from **Cauchy(0, 1)**.
- Don't worry about where this comes from; this is just an example

```
set.seed(2025) # Set seed for reproducibility

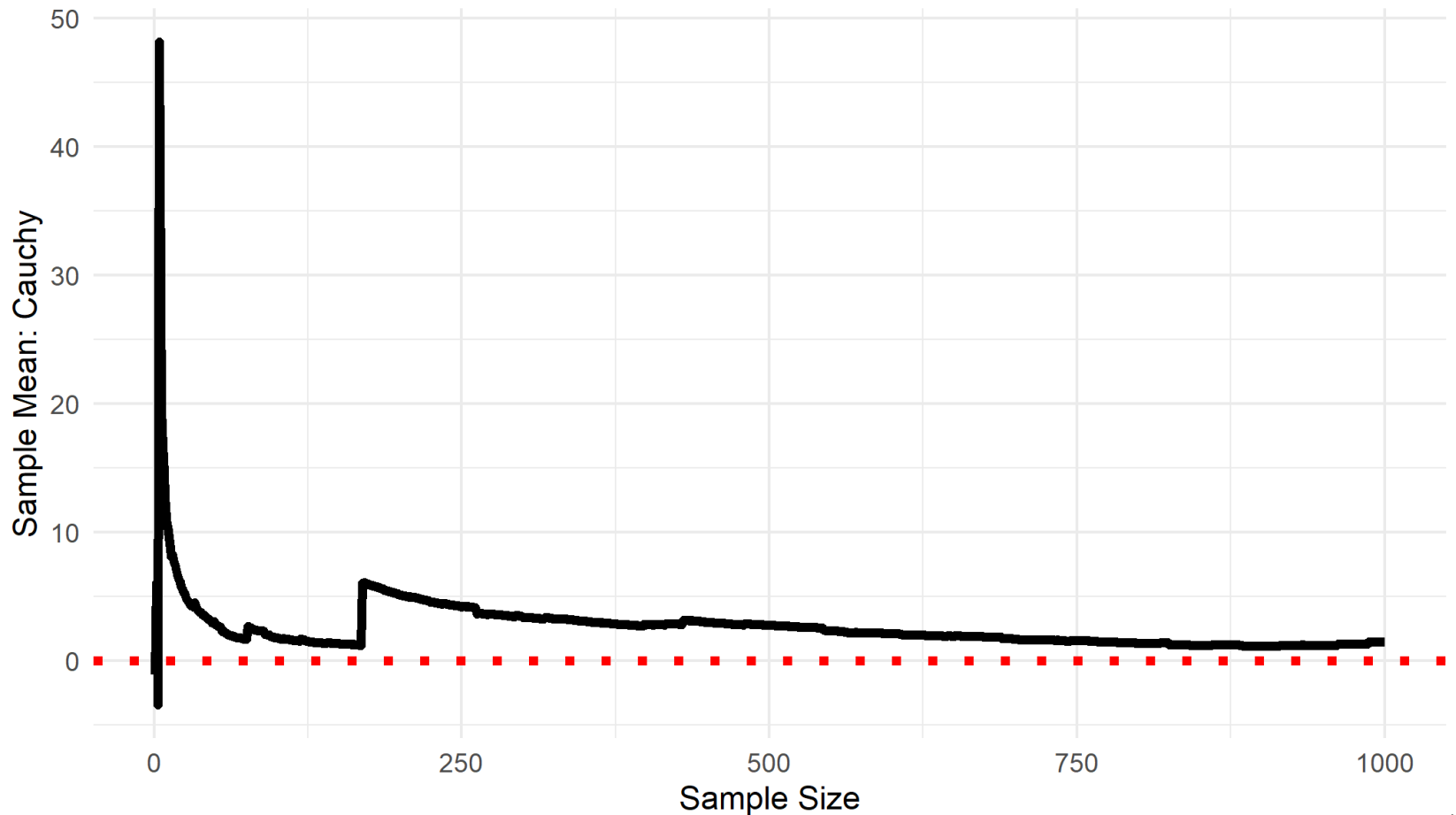
Nsim          = 1000                                # Number of simulations / sample size
samp_means    = cumsum(rcauchy(Nsim)) / (1:Nsim)      # Cumulative average of Cauchy draws

# Create data table for plotting
plot_data = data.table(x = 1:Nsim, y = samp_means)

# Plot the "sample mean" of Cauchy distribution (note: LLN does not hold)
LLN_plot3 = ggplot(plot_data, aes(x=x, y=y)) +
  geom_line(size=1.5) + ylab("Sample Mean: Cauchy") + xlab("Sample Size") +
  geom_hline(yintercept=0, linetype='dotted', col = 'red', size=1.5) +
  theme_minimal()
```

LLN Example 3

- Huh, doesn't appear to work!
- The Cauchy distribution is one of the distributions where the LLN breaks down.



Review of CLT

- Before finally examining a Monte Carlo simulation, we need to review the **Central Limit Theorem (CLT)**.
- While the LLN tells us what the **sample mean** converges to, the CLT tells us the **distribution of the sample mean** converges to.
- An estimator is a function that takes in random variables and spits out an estimate.
- As such, **estimators are random variables too! We call the distribution of the estimator "Sampling Distribution"**.
- The idea is that the sample mean is a function of random data and is thus random itself.
 - If you took another sample, the sample mean would be slightly different.
 - Picture doing this many times; the CLT tells us what this distribution will be.
- If $\hat{\mu}$ is the sample mean, the CLT tells us that

$$\hat{\mu} \sim^A N(\mu_X, \sigma_X^2/n)$$

- Note:
 - $E[\hat{\mu}] = E[\frac{1}{n} \sum_{i=1}^n X_i] = \frac{1}{n} \sum_{i=1}^n E[X_i] = \mu_X$
 - $Var[\hat{\mu}] = Var[\frac{1}{n} \sum_{i=1}^n X_i] = \frac{1}{n^2} \sum_{i=1}^n Var[X_i] = \frac{\sigma_X^2}{n}$

Monte Carlo Simulation

- The idea behind Monte Carlo (MC) simulation is that you can generate data from a DGP, estimate the parameters you're interested in, and then repeat a bunch of times.
- If your estimator "works," you should get a nice, normal distribution that matches the CLT.
- With MC, important to keep two different n 's separate in your head:
 1. The **sample size** which is the n that corresponds to the CLT previously.
 2. The **number of simulations** which is how many times the MC simulation is repeated.
- To reiterate: the MC simulation algorithm, at a broad level is as follows:
 1. Generate data by simulating the DGP with a sample size of N_{samp} ,
 2. Estimate the parameters of your model,
 3. Store the estimates,
 4. Go back to step 1 until you've repeated it N_{sim} times.

Monte Carlo for the Sample Mean

Let's run an MC simulation for the sample mean for data drawn iid from $U(0, 1)$

```
set.seed(2025)                                # Set seed for reproducibility

Nsim      = 1000                               # Number of Monte Carlo simulations
Nsamp     = 100                                # Sample size for each simulation
sample_means = rep(0, Nsim)                    # Preallocate vector to store sample means

# Run simulations
for(sim in 1:Nsim){
  draws = runif(Nsamp)                         # Draw Nsamp values from Uniform(0,1)
  sample_means[sim] = mean(draws)              # Compute sample mean
}

# Create density curve for theoretical distribution of the sample mean
xs = seq(min(sample_means), max(sample_means), length.out = 100)
ys = dnorm(xs, mean = 1/2, sd = sqrt(1/12/Nsamp)) # Normal approximation using CLT

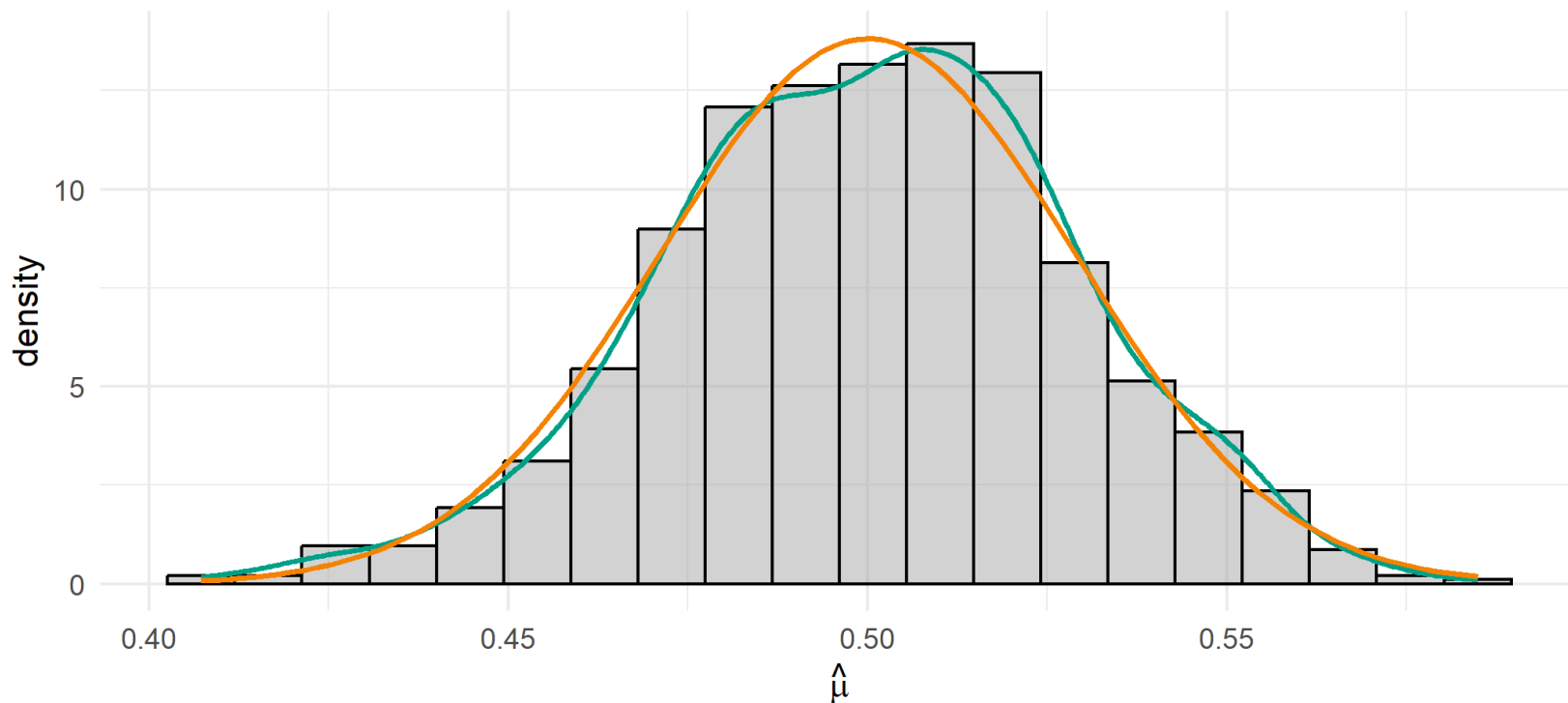
# Store density data in a data.table for plotting
den_data = data.table(x = xs, y = ys)

# Store Monte Carlo sample means for plotting
MC_data = data.table(x = sample_means)
```

Monte Carlo for the Sample Mean

Monte Carlo Simulation for Sample Mean: Nsamp = 100

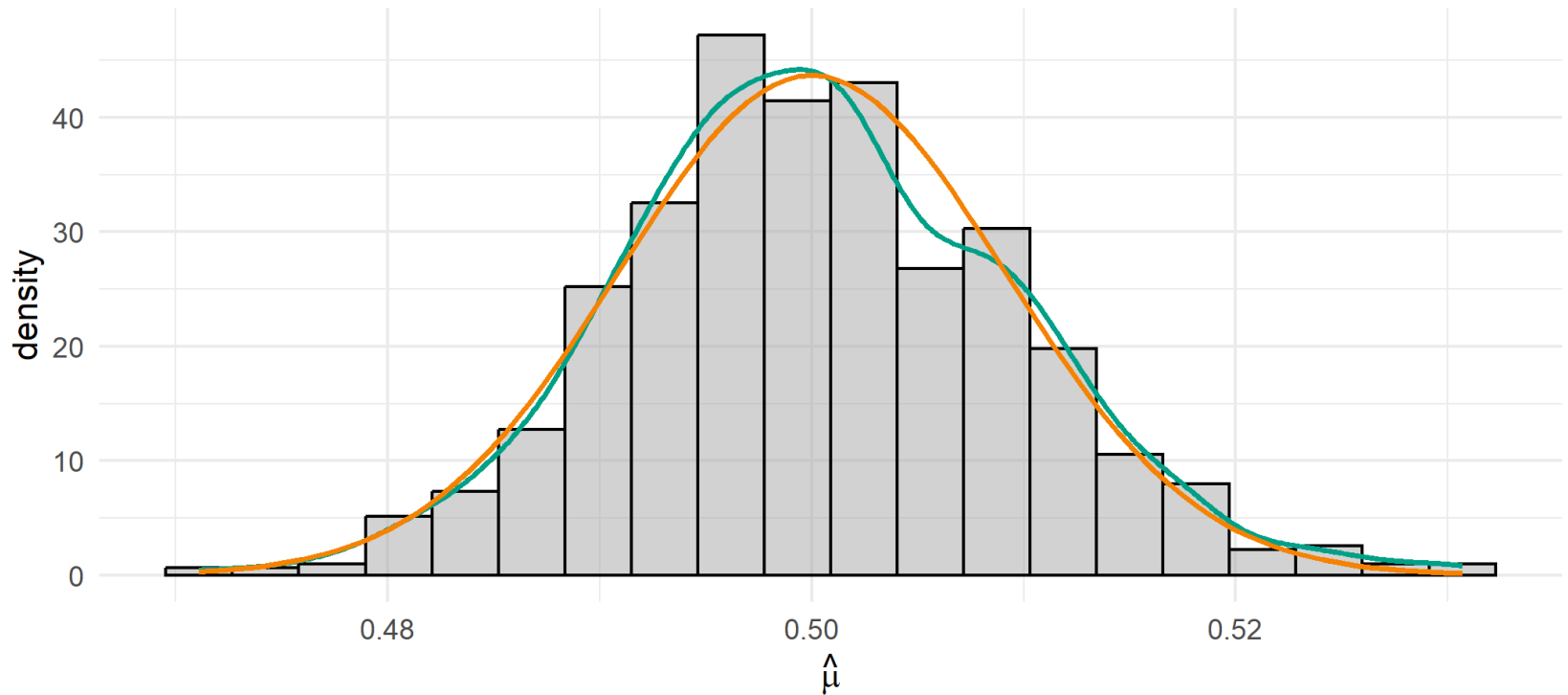
Densities  Empirical Density  Theoretical Density  Histogram



Repeat with Nsamp=1000

Monte Carlo Simulation for Sample Mean: Nsamp = 1000

Densities Empirical Density — Theoretical Density Histogram



MCs for Other Estimators

- We can use MC simulation with any (consistent) estimator
- Let's try the two estimator's for the sample variance:

$$\frac{\sum (x_i - \bar{x})^2}{N} \text{ vs. } \frac{\sum (x_i - \bar{x})^2}{N - 1}$$

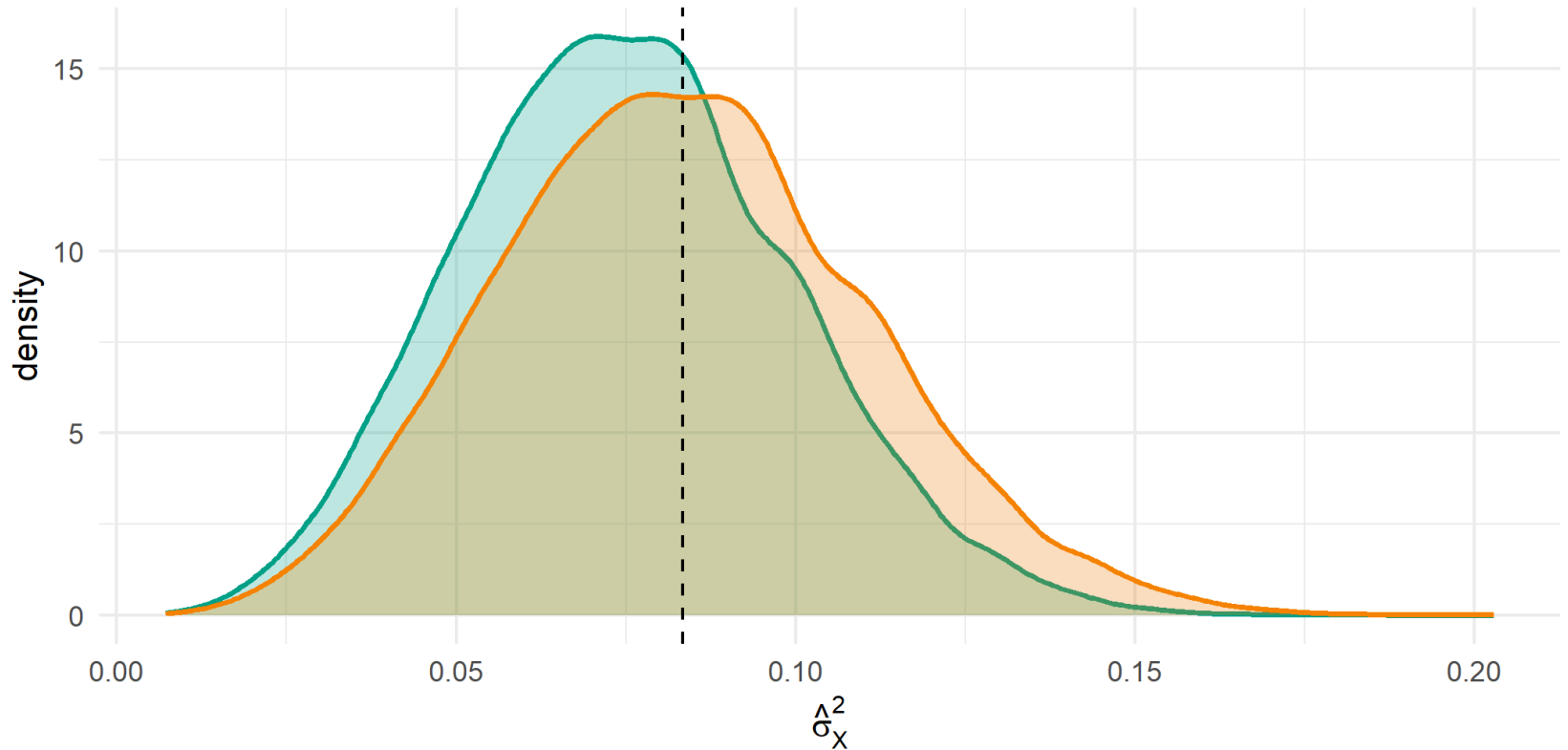
- The theoretical distribution is tedious to derive because you need the theoretical variance of the sample variance (confusing!), but we can still use MC to illustrate a point.

```
set.seed(2025)
Nsamp      = 10           # set sample size
Nsim       = 10000        # set number of MC simulations
sample_varN = rep(0,Nsim) # preallocate vector to store MC estimates
sample_varNm1 = rep(0,Nsim) # preallocate vector to store MC estimates

for(sim in 1:Nsim){
  draws = runif(Nsamp) # draw Nsamp values from U(0,1)
  sample_varN[sim] = mean((draws-mean(draws))^2) # sample var dividing by 1/N
  sample_varNm1[sim] = var(draws) # sample var dividing by 1/(N-1)
}
```

MC Distribution for Sample Variances

Estimator █ N █ N-1



First Look at the Bias Variance Trade-Off

```
##          groups      mean      variance
##      <char>      <num>      <num>
## 1:           N 0.07513191 0.0005860201
## 2:          N-1 0.08347990 0.0007234816
## 3: Theoretical 0.08333333 0.0000000000
```

If we calculate variance with $\frac{\sum (x_i - \bar{x})^2}{N}$ compared to $\frac{\sum (x_i - \bar{x})^2}{N-1}$, we are further away from the true value (more biased) but the estimates are more stable (less variance).

Inference vs Prediction

A reminder that in Lecture 9, we get estimators $\hat{\beta}$ for the linear model β (ignoring constant for simplicity):

$$y_i = \beta x_i + e_i$$

One approach to estimation is to choose an estimator that minimizes the "mean-squared error" (MSE)

- The MSE is the average squared difference between the estimate $\hat{\beta}$ and the true value β

$$\text{MSE}_{\beta}(\hat{\beta}) = E[(\hat{\beta} - \beta)^2]$$

However, different estimators can minimize the MSE but have different properties. Using algebra, we can write the MSE like so:

$$\text{MSE}_{\beta}(\hat{\beta}) = \text{Bias}(\hat{\beta}, \beta)^2 + \text{Var}(\hat{\beta})$$

This decomposition of the MSE encapsulates the different goals of traditional statistics (inference) and machine learning (prediction).

OLS being BLUE

Probably you have learned that OLS is **BLUE** (best linear unbiased estimator). Let's demonstrate here with simulation. Suppose the DGP is:

$$y_i = 0x_i + e_i$$

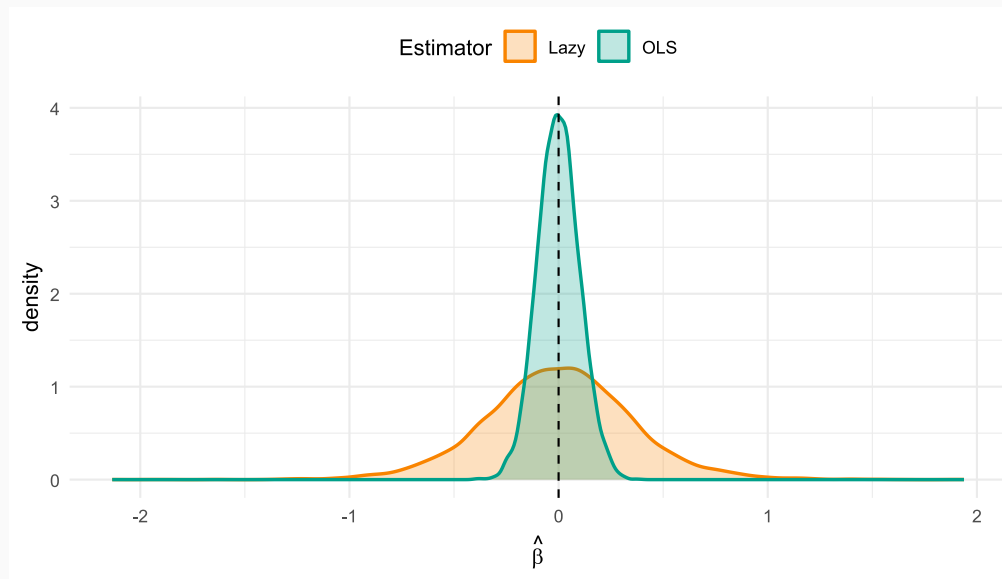
where both x_i and e_i are i.i.d. normal.

Suppose our sample size is 100, and We can have two estimators of β :

- one is regular OLS, $\hat{\beta}_{OLS}$
- the other is a lazy version that I will simply use the first 10 observations to get the OLS estimate, $\hat{\beta}_{lazy}$.

Let's do the 1000 simulation and compare the estimates. The theory says among all linear unbiased estimators, OLS estimates have the smallest variance. Let's see if it is true in our simulation.

OLS being BLUE



You can see that although the lazy estimates get the value right on average (no bias), but it has much bigger variance (the estimator generally has larger errors).

Bias Variance Tradeoff

Traditional statistics $\rightarrow$ inference of β

- We want to try to infer the value of β from the data.
- Choose an estimator $\hat{\beta}$ that has low variance and low bias to perform "powerful" statistical tests and make conclusions about the value of β .
- Usually (though not always) this requires us to choose an unbiased estimator so we aren't "skewing" the estimates $\hat{\beta}$ too much.

Machine Learning $\rightarrow$ predicting y

- Estimate some model given "training data" and then predict the value of y given a new value of x .
- Call this prediction $\hat{y}$. In our example, $\hat{y} = x\hat{\beta}$
- Usually want the variance of the prediction to be small (noisy predictions are useless).
- In our example, given a new x , we have $Var(\hat{y}) = Var(x\hat{\beta}) = x^2 Var(\hat{\beta})$.
- To decrease the variance of $\hat{y}$, we must decrease the variance of $\hat{\beta}$.
- Leads to choosing estimators with some bias if it decreases the variance enough.

Bootstrap

Bootstrapping uses resampling technique to generate the sampling distribution of a estimator. It can be used for multiple purposes. If you are interested, you can check more on this book:

- Davison, A. C. & Hinkley, D. V. (2009). *Bootstrap methods and their application*. New York, NY: Cambridge University Press.

In this lecture, I will only talk about the spirit and basic use of bootstrap.

Bootstrap Idea

The idea of **sampling distribution** would still be the same:

Each random draw of sample j -> estimator j (e.g. $\hat{\beta}^j$)

This is perfectly: $\hat{\beta}^j$ is a random variables! Collecting those estimators and plot them, we will have the sampling distribution!

In **Monte Carlo Simulation**, we can easily generate new sample based on assumed DGP.

In real world, ideally, we want to draw more samples (conduct more surveys), do estimates for each sample, and plot the sampling distribution. However, what if we are limited to what we have?

We can **sample our sample**! This is the spirit of **bootstrap**!

Residual Bootstrap

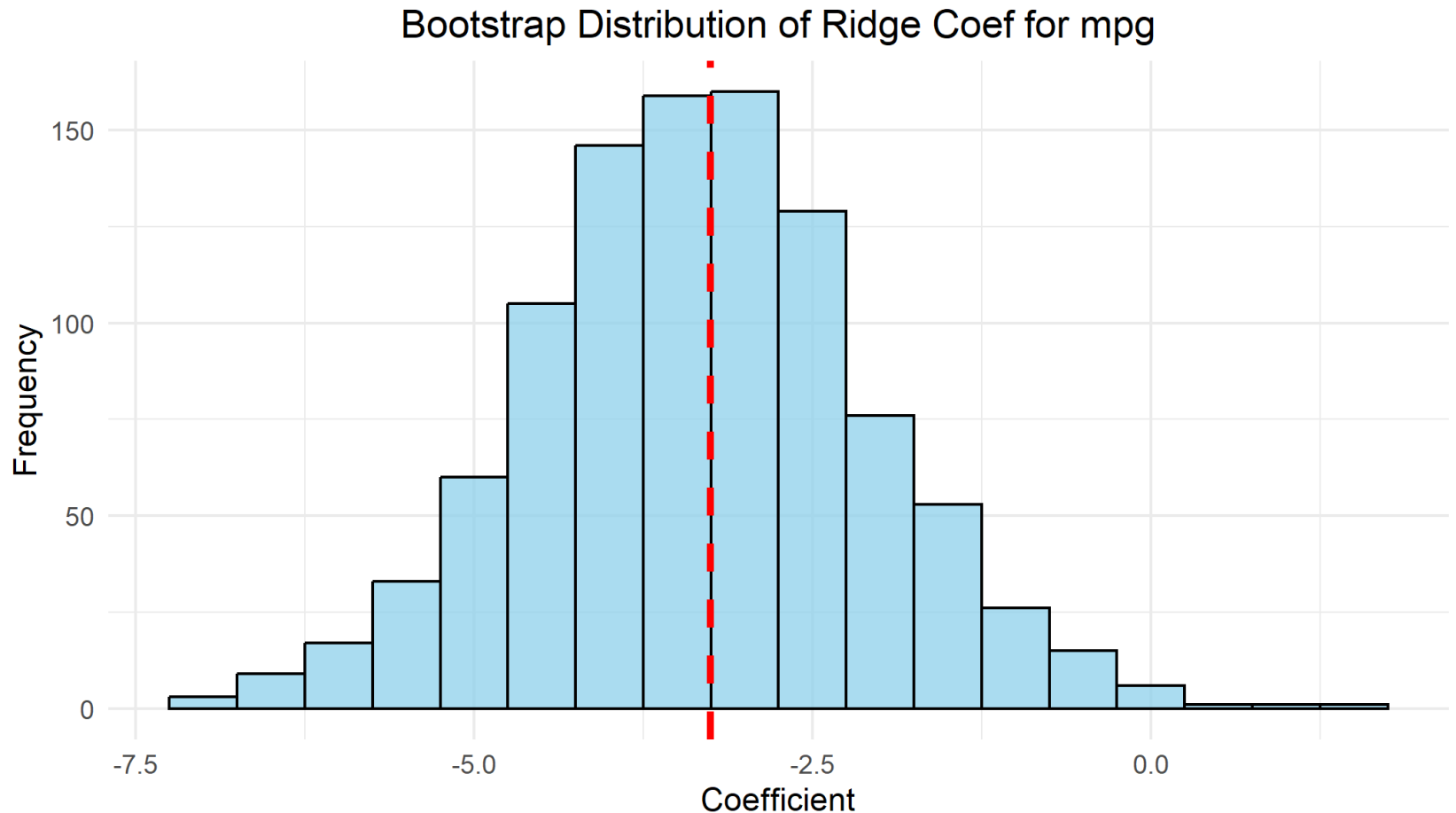
There are many different types of bootstrap techniques. If we believe the error term is nice, we can use **residual bootstrap**. (If there are heteroskedasticity, you should use **wild bootstrap**.)

Here are the steps for a residual bootstrap:

1. Fit a model (as in lecture 9).
2. Treat the fitted value, $\hat{y}$, as the true expected value without noise and Residual, e , as true noise. Randomly reassign the "noise" to the "expected value" to create new $y = \hat{y} + rand(e)$.
3. Rerun the model using the new "sample".
4. Repeat steps 2 and 3 for B times.

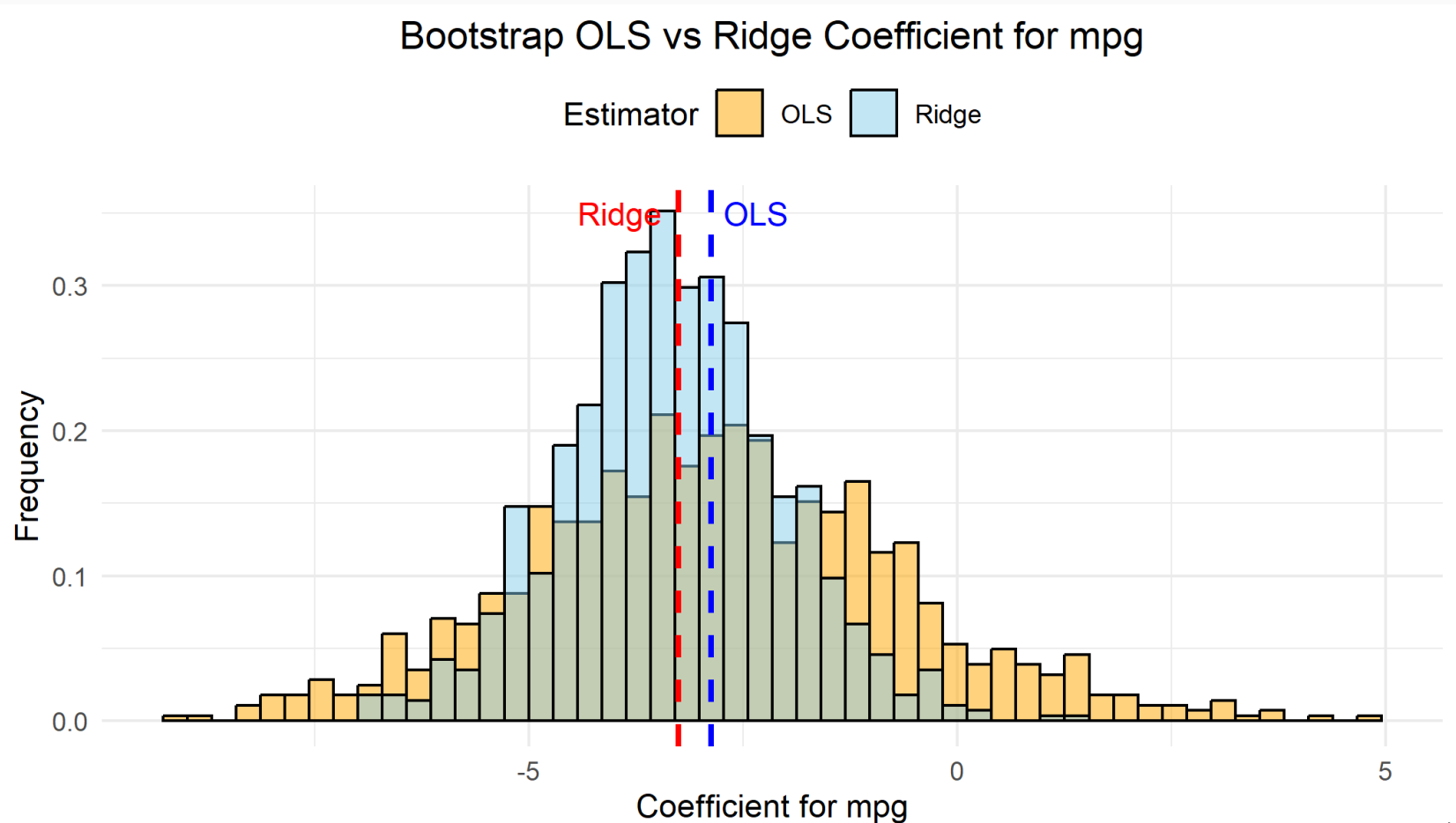
Residual Bootstrap for Ridge

Let's use mtcars data again to do the Ridge regression like in lecture 9. Suppose that we want to use $\lambda = 5$:



Bias Variance Tradeoff

We can see the bias variance tradeoff with real data! Again, we can bootstrap the OLS estimates sampling distribution and plot them together.



Descriptive Statistics and Tests: LM

LM Summary

We have seen many results from the summary of `lm`. We were just using the estimates, and now, let's take a look at the rest:

```
##
## Call:
## lm(formula = hp ~ mpg + wt + drat + qsec, data = mtcars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -48.801 -16.007  -5.482  11.614  97.338
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  473.779    105.213   4.503 0.000116 ***
## mpg         -2.877      2.381   -1.209 0.237319
## wt          26.037     13.514    1.927 0.064600 .
## drat         4.819     15.952    0.302 0.764910
## qsec        -20.751      3.993   -5.197 1.79e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 32.25 on 27 degrees of freedom
## Multiple R-squared:  0.8073,    Adjusted R-squared:  0.7787
## F-statistic: 28.27 on 4 and 27 DF,  p-value: 2.647e-09
```

Inference - Std. Error

Std. Error stands for standard error, whose value is from a theoretical formula in the background. It is the standard deviation (square root of variance) of the **sampling distribution**.

In fact, we can use **bootstrap** to show this point:

```
## [1] 2.153514
```

So, based on bootstrap result, if we "draw 1000 random sample" and conduct OLS on each sample, we will have the estimated mpg coefficient following a distribution whose standard deviation is about 2.153 (or based on the formula with some assumptions: 2.381).

Inference - t value

t value stands for t-statistics from Student T Distribution. This is a standardized statistics.

Based on CLT, we know that the sampling distribution almost follows a normal distribution, which means if we standardize with the correct mean μ and σ , we should have:

$$\hat{z} = \frac{\hat{\beta} - \mu}{\sigma} \sim N(0, 1)$$

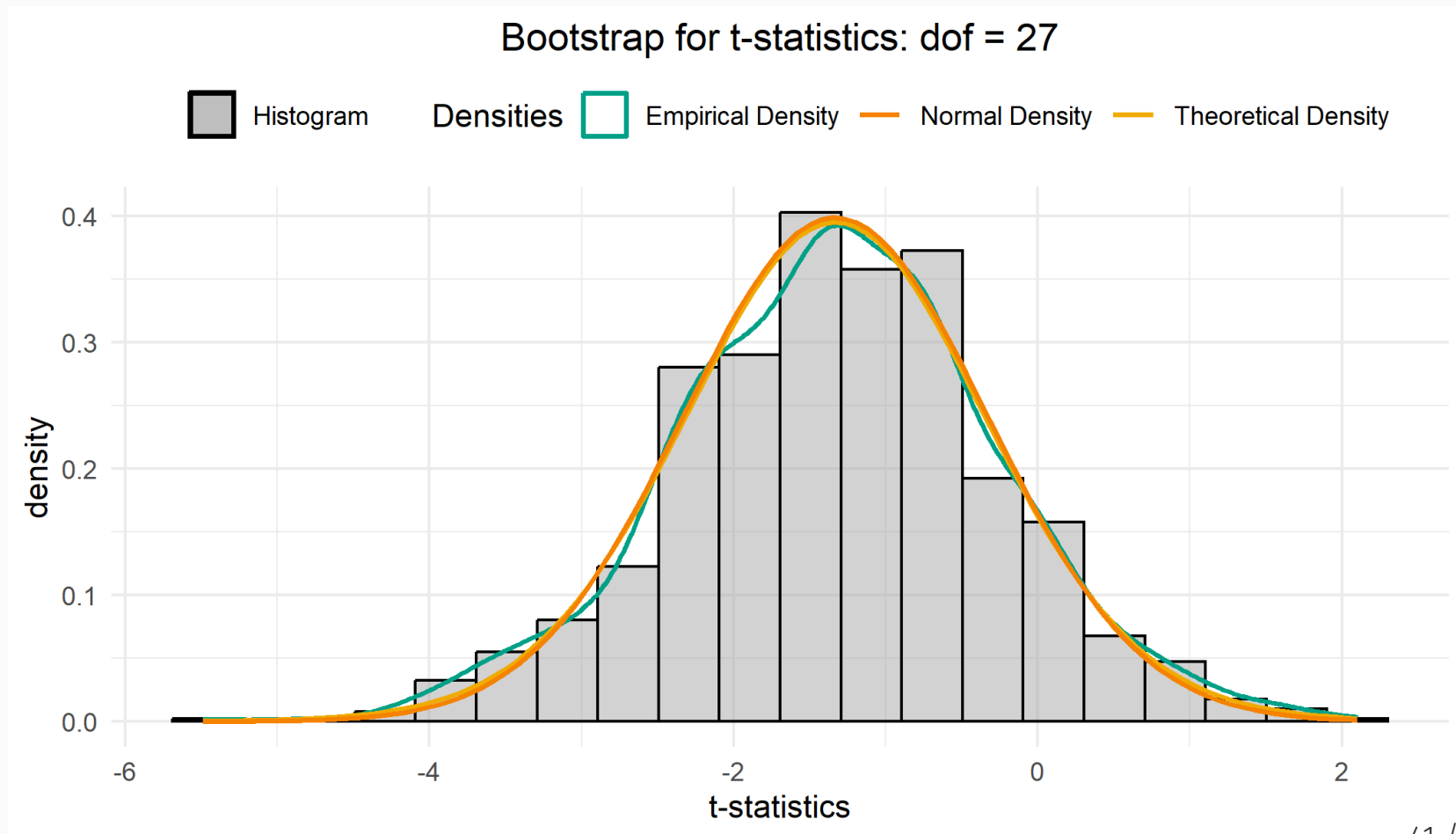
However, the major problem is: we do **NOT** know the true σ ! The best we can use is **Std. Error** (used by `lm`) or the bootstrap estimates. There is a cost to it by using imprecise $\hat{\sigma}$. Now it follows a t-distribution instead of normal distribution:

$$\hat{t} = \frac{\hat{\beta} - \hat{\mu}}{\hat{\sigma}} \sim t(d.o.f.)$$

Also, based on LLN, the larger the sample is, the closer $\hat{\sigma}$ is to true σ ! So the sample size matters! This is why we have **degrees of freedom** for t-distribution ($n - k - 1$ for linear regression where k is the number of parameters.)! As you would imagine, the higher d.o.f. is, the closer t-distribution is to normal distribution!

Bootstrap the t-statistics

Now, let's plot the distribution of t-statistics and overlay with a t-distribution with $d.o.f. = n - 1$.



Inference - $\Pr(>|t|)$ and Signif. codes

$\Pr(>|t|)$ is the p-value. It has a specific meaning based on t-test of $\beta = 0$. Suppose $p\text{-value} = 0.002$:

If the null hypothesis is true (i.e. the true $\beta = 0$), we have **0.2%** probability to draw our sample in hand or something more extreme.

So a **low p-value** or a **high t-statistics** means that we are pretty unlikely to draw our sample in hand if the $\beta = 0$. This gives reasonable conclusion that maybe $\beta \neq 0$ (we can never be **100%** sure though).

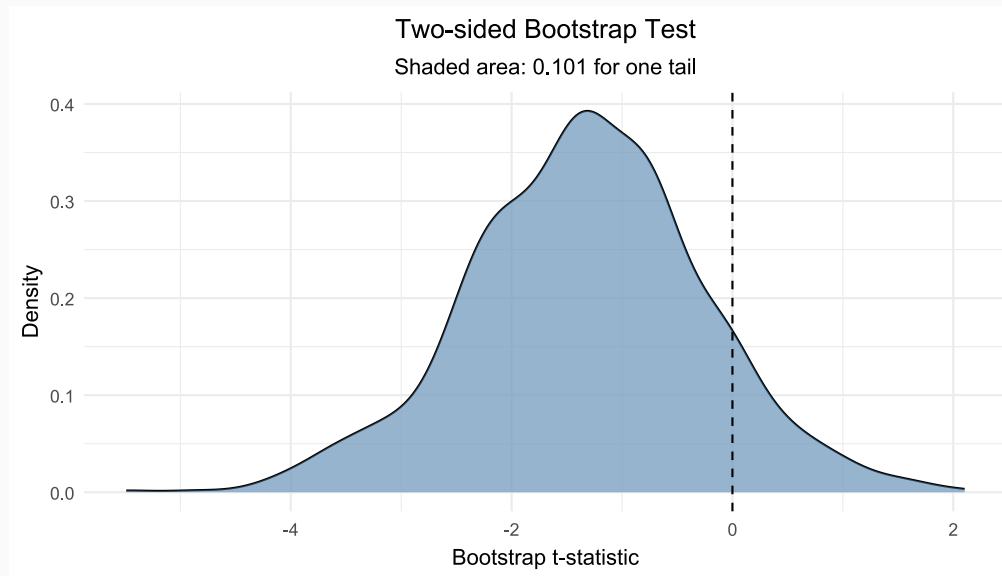
For hypothesis testing, we set a confidence level, denoted as α , usually at **1%**, **5%**, or **10%**. If $p\text{-value} < \alpha$, we state that "there is statistically significant evidence at the α level that β is not 0."

The **Signif. codes** are:

- *** for significant at **0.1%** level
- ** for significant at **1%** level
- * for significant at **5%** level
- . for significant at **10%** level

Bootstrap and p-value

We can get similar results if we use the bootstrap results. Since we have a two-sided test, the p-value is twice the chance $t \geq 0$



```
## [1] "The two-sided p-value for beta = 0 from bootstrap is 0.2."
```

```
## [1] "The two-sided p-value for beta = 0 from lm is 0.24."
```

Again, they are pretty close! The large p-value suggests that we do not have enough evidence to say "mpg" is associated with "hp".

Overall - F-statistics and p-value

F-statistics is a joint hypothesis test. The null hypothesis is that all β 's are 0.

We won't go over the math, but p-value has similar meaning for all tests. Here, we have "p-value: 2.647e-09". This is below 0.01, which means "there is statistically significant evidence at the 1% level that β are not all 0."

Overall - R-squared

Remember that we have a loss function MSE:

$$MSE = \frac{1}{N} \sum_i^N e_i^2$$

where e is the residual (unexplained part) of the model.

Can we compare model goodness of fit across different data?

No. Suppose we have return at decimal level vs. returns at percentage level, we will have MSE to be 100x difference! The two models are essentially the same though! How do we deal with that?

Remember the standardizing idea, we can standardize the errors by divided by variance of y :

$$\frac{MSE}{Var(y)} = \frac{\sum_i^N e_i^2}{\sum_i^N (y - \bar{y})^2}$$

Overall - R-squared

These are the bad part that the model cannot explain, let's put in a more positive tone by creating the R^2 . **Multiple R-squared** has the formula:

$$R^2 = 1 - \frac{MSE}{Var(y)} = 1 - \frac{\sum_i^N e_i^2}{\sum_i^N (y - \bar{y})^2}$$

R^2 is a **goodness of fit** measure. For example, $R^2 = 10\%$ means that variation of our model prediction can explain 10% actual variations in y . However, in `lm`, it is an **in-sample** measure!

If you are increasing the number of explanatory variables, the **in-sample R-squared** will always increase, so someone take this into consideration and created the **Adjusted R-squared**:

$$R_{adj}^2 = 1 - \frac{n - 1}{n - k - 1} \frac{\sum_i^N e_i^2}{\sum_i^N (y - \bar{y})^2}$$

This is a very heuristic adjustment though. Nonetheless, this is a first attempt to make the model work better out-of-sample. In the later session, we will see how to test and compare models **out-of-sample**.

Training/Testing: LM

Prediction Out-of-Sample (OOS)

Out-of-Sample (OOS) Performance:

- OOS is mostly about **prediction**, but also matters for **inference**: we want $\beta_{\text{test}} \approx \beta_{\text{train}}$.
- We can compare models in-sample with R^2 or R^2_{adj} , but it is better compare models OOS.
- Adding more variables increase in-sample R^2 , but may overfit -> poor OOS performance.

Overfitting analogy:

- Student memorizes practice exam (good in-sample) -> fails when questions change (bad OOS).

How to prevent/check:

1. Split data: Training / Testing (cross-section) or rolling/expanding windows (time-series)
2. Fit model on training data
3. Evaluate on testing data

Illustration with OLS: Data Splitting

Let's use `mtcars` again and try to predict/explain "hp". It seems that p-value for "drat" is really large. It might not be a "useful" variable (in terms of regression). Let's use the idea of training and testing to check this formally:

- Model 1 uses "mpg", "wt", "drat", "qsec"
- Model 2 uses "mpg", "wt", "qsec"

We can split data in different proportions. Here, we will use one common split: 70/30 training/testing.

```
# Split data randomly
set.seed(2025) # do not forget to set seed as randomness is introduced!
# Note that we do not want replacement here. The entire testing data should not be seen
Ind_train = sample(x = 1:nrow(mtcars),
                   size = as.integer(nrow(mtcars)*0.7))

# Extract the training and testing data
Training_cars = mtcars[Ind_train,]
Testing_cars = mtcars[-Ind_train,]

# Model Fitting
Model1 = lm(Training_cars$hp ~ ., Training_cars[,c("mpg", "wt", "drat", "qsec")])
Model2 = lm(Training_cars$hp ~ ., Training_cars[,c("mpg", "wt", "qsec")])
```

Illustration with OLS: OOS Testing

There are various metrics to assess the model performance, such as **MSE**, R^2 , but also **MAE** (mean absolute error, $\frac{1}{N} \sum_{i=1}^N |e_i|$) and **RMSE** (Root MSE, $\sqrt{MSE}$). Let's use **RMSE** here:

```
## [1] "IS R^2 for Model 1 is 0.761; Adjusted R^2 is 0.705"
```

```
## [1] "IS R^2 for Model 2 is 0.76; Adjusted R^2 is 0.72"
```

```
## [1] "OOS RMSE for Model 1 is 30.75"
```

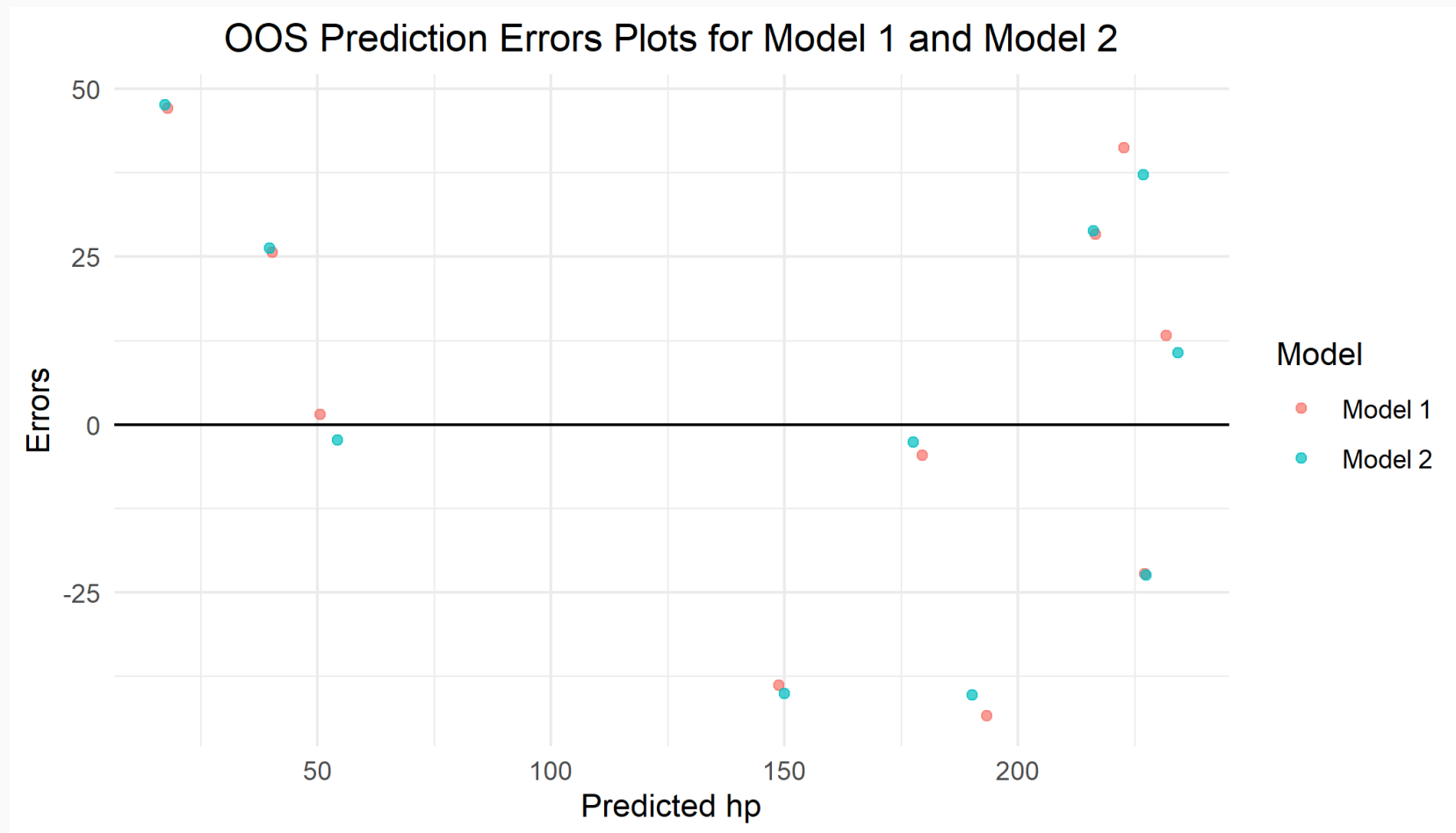
```
## [1] "OOS RMSE for Model 2 is 30.04"
```

In sample, R^2 suggest we should adopt Model 1 but R_{adj}^2 suggest we should adopt Model 2.

However, based on **OOS RMSE**, Model 2 yields a much better prediction than Model 1. Not include "drat" is better (in prediction sense). The **OOS** result should be trusted here!

Illustration with OLS: OOS Error Plots

Now, let's plot predicting errors.



Simply comparing the numbers is just a heuristic, though many people stop at this step. 51 / 69

OOS Performance Tests

The gold standard for predictive accuracy comparisons is **Diebold-Mariano test** (This test can also be applied to testing data in time-series splits.). You will find that this test is also developed based on the idea of CLT.

Each random draw of sample $\rightarrow$ per unit OOS loss function value for Model 1, $L1_i$

Each random draw of sample $\rightarrow$ per unit OOS loss function value for Model 2, $L2_i$

This is perfectly: both $L1_i$ and $L2_i$ are random variables!

Ideally, if we see for every i , $L1_i \geq L2_i$, we can be certain that Model 1 is better than Model 2 (in terms of prediction)!

However, what if we see some mixed result?

OOS Performance Tests

We can define the difference $d_i = L1_i - L2_i$. $d_i \leq 0$ is equivalent to $L1_i \leq L2_i$. Note that since our goal is to establish $L1_i > L2_i$ (Model 2 is better), we need to put it in the alternative hypothesis. Our null hypothesis is $d_i \leq 0$.

By CLT again, d_i follows a **normal** distribution. However, since we need to standardize with estimated variance, we need t-test (again):

```
# loss differences
d_i = (Error1)^2 - (Error2)^2

# t-test on loss differences
t.test(d_i, alternative = c("greater"))

##
##      One Sample t-test
##
## data:  d_i
## t = 1.008, df = 9, p-value = 0.1699
## alternative hypothesis: true mean is greater than 0
## 95 percent confidence interval:
##  -35.41043      Inf
## sample estimates:
## mean of x
```

Training/Validation/Testing: Ridge

Validation for Hyperparameters

Comparing the numbers alone is just a heuristic, yet many stop here.

Now, let's look at another unsolved mystery. Many machine learning methods come with hyperparameters (for instance, the λ in Ridge). How do we pick them?

In principle, we'd like the λ that gives the best OOS performance.

So can we simply split the data into training/testing, try a menu of λ 's, fit Ridge on the training set for each one, choose the λ with the lowest test MSE, and celebrate?

No. That would be like peeking at the answer key while practicing.

The test set must remain untouched until the very end.

So what's the right workflow?

Validation for Hyperparameters

We split the training data once more into **training** and **validation** sets.

The validation set becomes a kind of rehearsal stage, a “pseudo-OOS” arena where we compare hyperparameters using fresh data that is still not the test set.

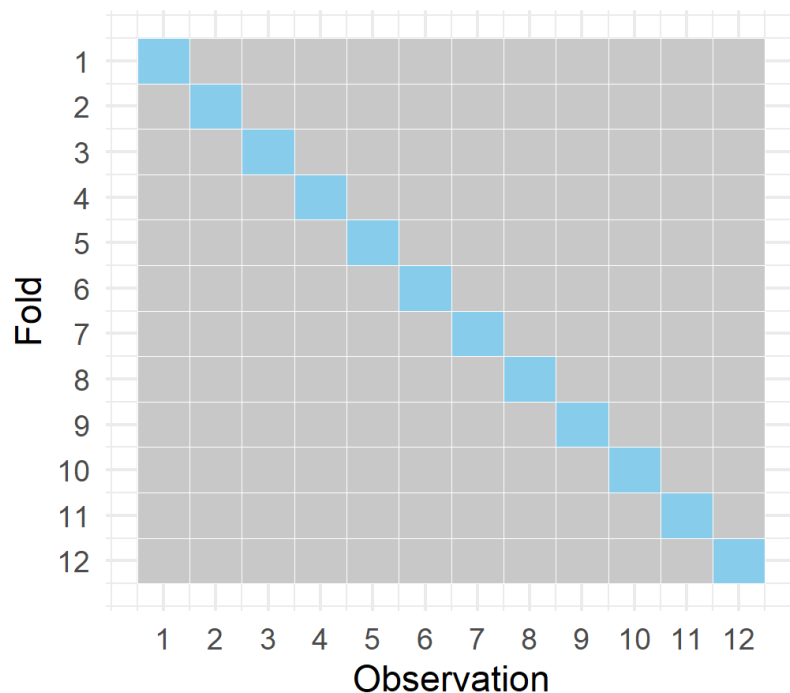
- Training is learning the material.
- Validation is trying mock exams to tune your strategy.
- Testing is the real final: one shot, no do-overs.

This process is known as "tuning", "fine tuning" or "validating".

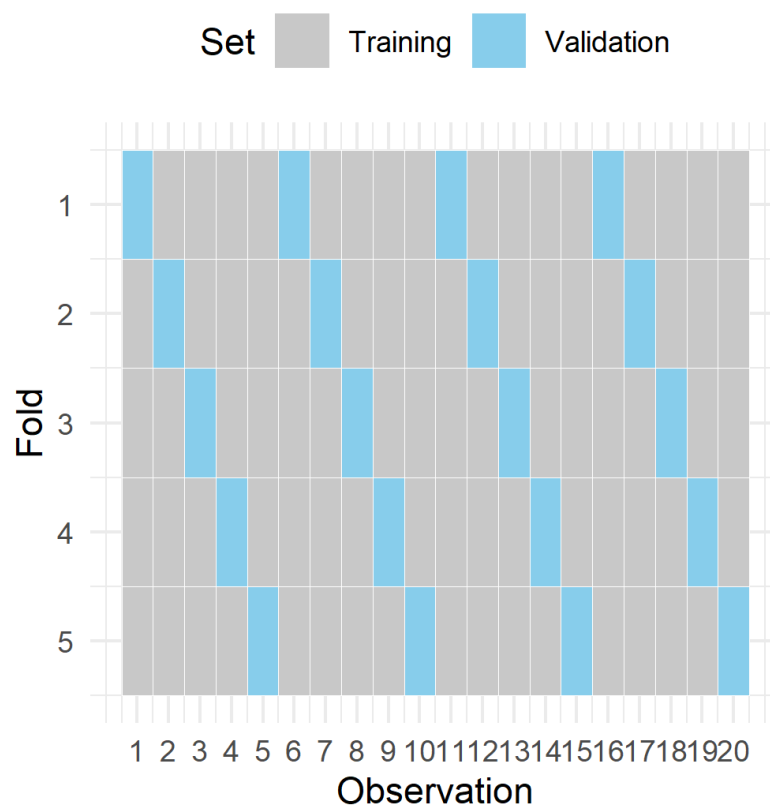
Cross Validation (Cross-sectional)

For cross-sectional data, instead of split training data into **training** and **validation** sets once, we can do it multiple times and find the best hyperparameter on average!

LOOCV



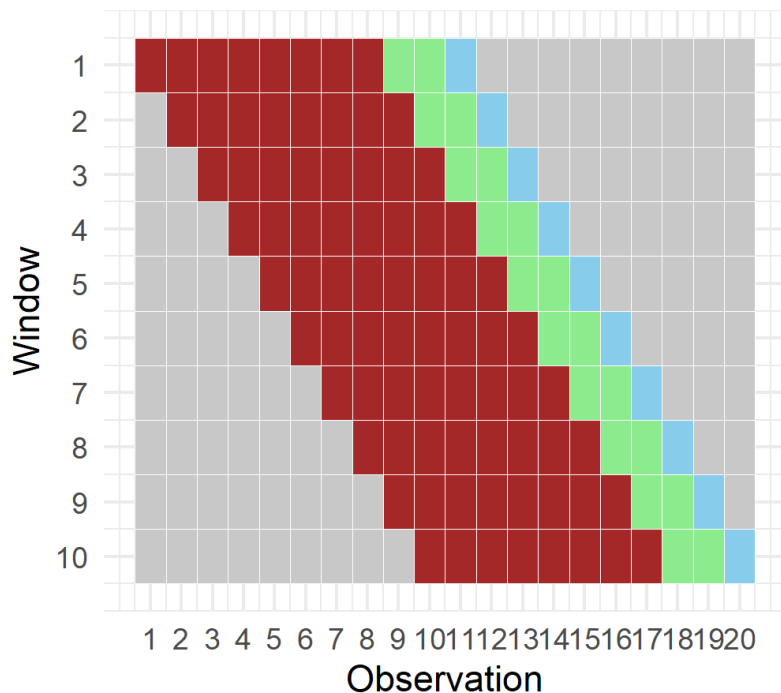
5-Fold CV



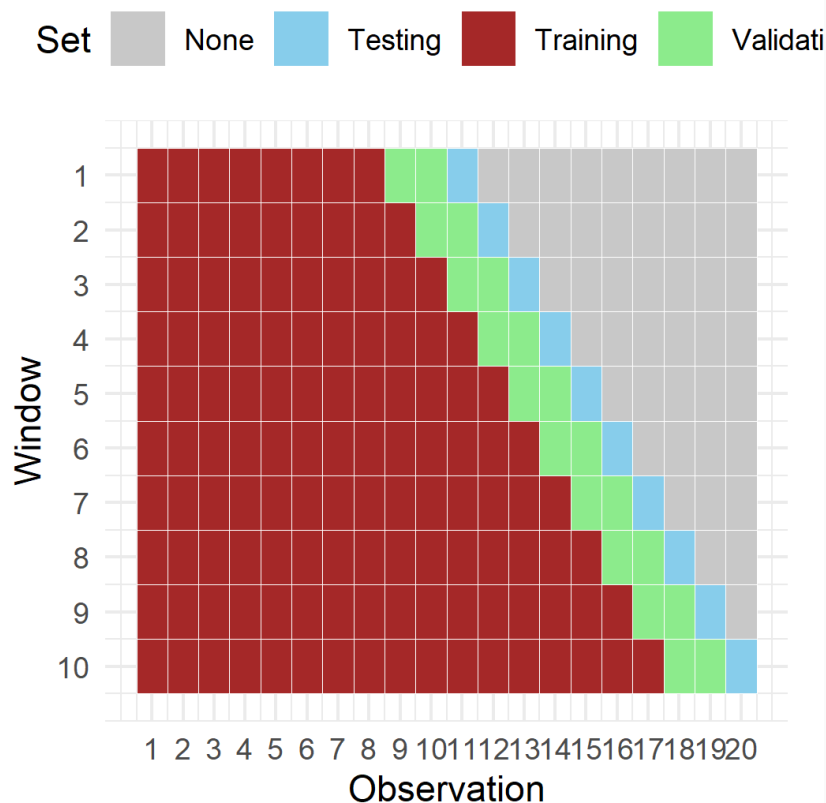
Cross Validation (Time-series)

One often needs to be careful when there is a time-dimension involved. The validation set must appear after the training data and test set needs to be even later!

Rolling-Window CV



Expanding-Window CV

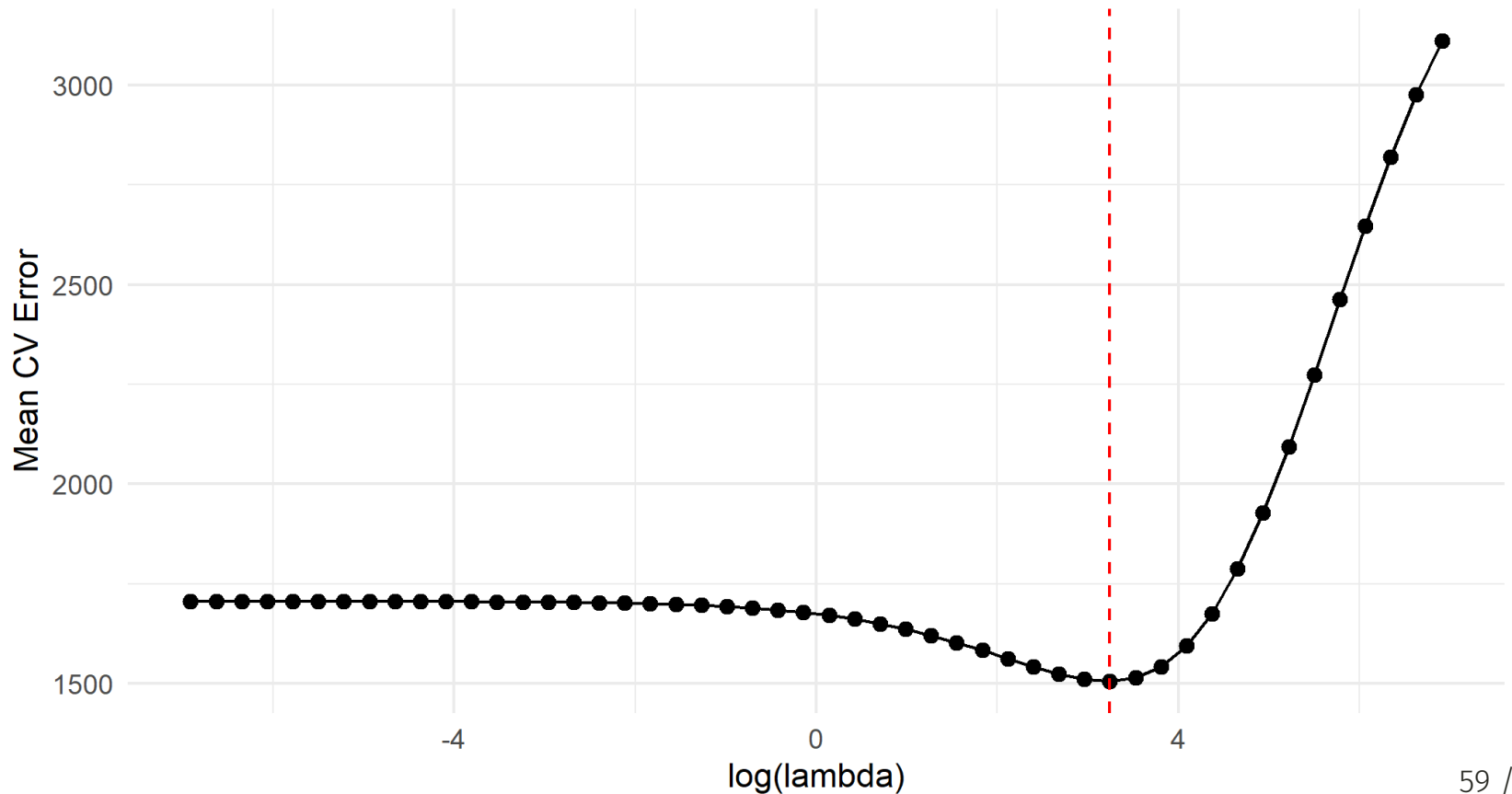


for loop

Again, we can use a grid search method with for loop to find the best λ !

```
## [1] "Best Lambda from 10-fold CV: 25.6"
```

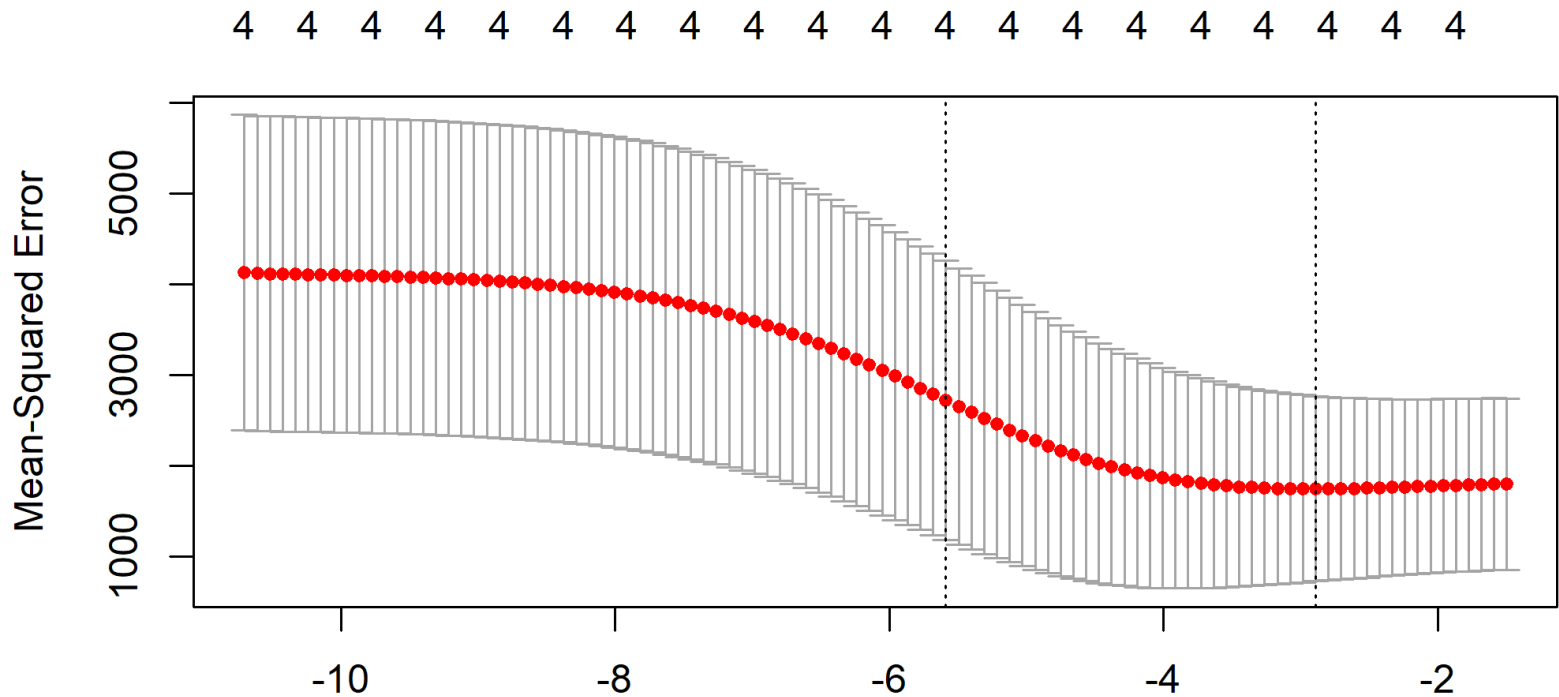
10-Fold CV for Ridge



glmnet Package

For functions in `glmnet` package, such as Ridge, LASSO, logistic Regression, etc. There are built-in CV's. Let's use Ridge as an example. Note that we have 10-fold CV as default. One can set `nfolds = nrow(x)` to archive LLOCV.

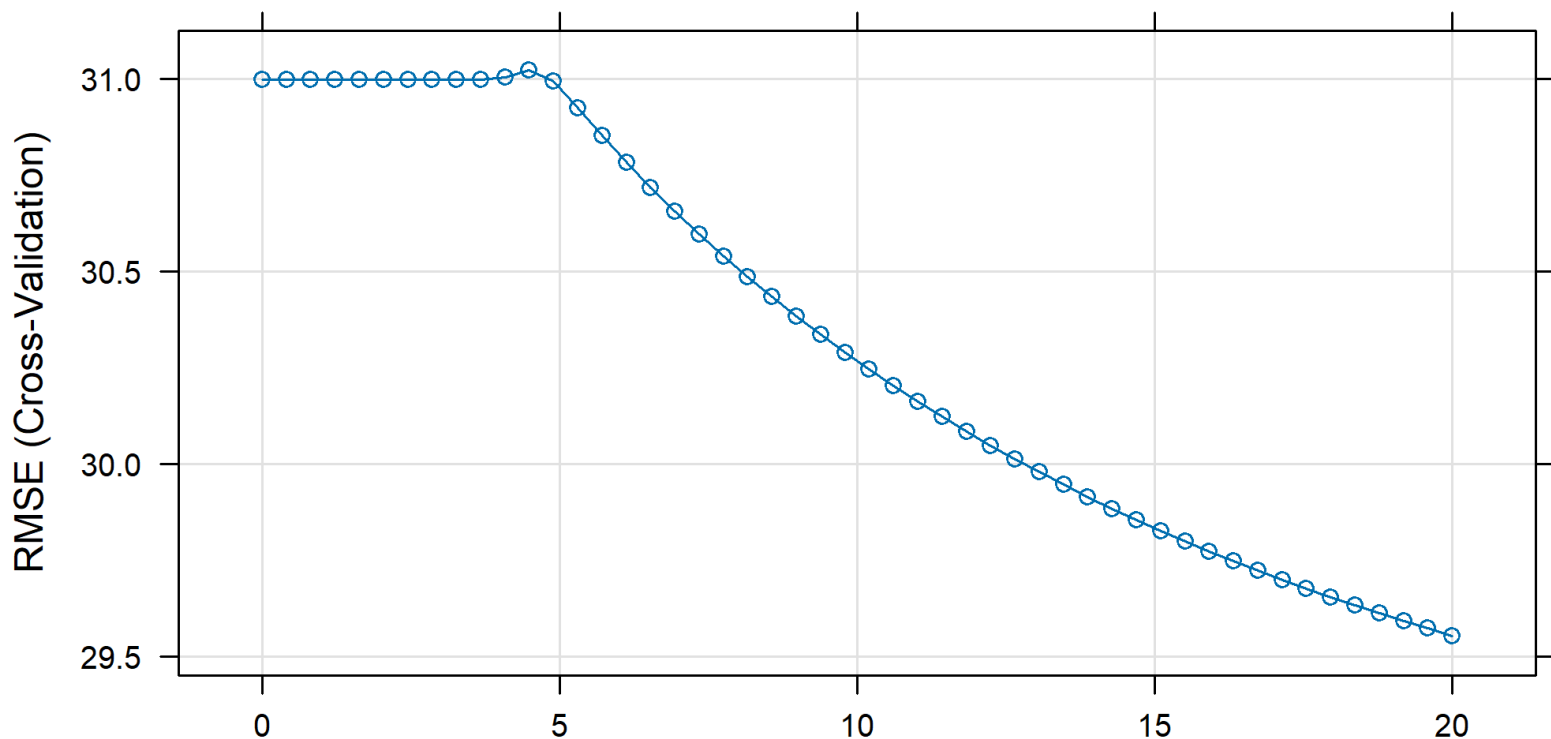
```
## [1] "Best Lambda from 10-fold CV: 18"
```



caret Package

The most general *R* package for cross-validation would be `caret` package. You can fix many models with this package (You can run `names(caret::getModelInfo())` to get a list of possible models). Let's look at Ridge as an example again:

```
## [1] "The best tuning result is: 20"
```



Refit the model with λ_{best}

Once you identify the best hyperparameters, you must refit the model on the combined training + validation data to re-estimate the parameters.

For example, suppose one of the tuning methods gave you the optimal value $\lambda = 20$, stored as `best_lambda`:

```
# x and y for the whole training + validation set
y = Training_cars$hp
x = data.matrix(Training_cars[, c("mpg", "wt", "drat", "qsec")])

# Refit with optimal lambda
ridge_best = glmnet(x, y, alpha = 0, lambda = best_lambda)
coef(ridge_best)

## 5 x 1 sparse Matrix of class "dgCMatrix"
##              s0
## (Intercept) 479.005789
## mpg        -3.823612
## wt         15.515874
## drat       -11.152344
## qsec       -15.018478
```

Time-Series CV

I would suggest to use for loop for CV in time-series modeling. Let's take our code from final presentation, but do a Ridge instead of OLS. The entire workflow is provided in Template file. As reminder, for our final project, this form of expanding window is **required**:

Expanding window: Have a 12-year window and 1 additional year for testing at a time, then expand training window by 1 year and everything else forward by 1 year:

- 1985-2000 as training, 2001-2008 as validation, and 2009 to evaluate the model
- 1985-2001 as training, 2002-2009 as validation, and 2010 to evaluate the model ...
- 1985-2015 as training, 2016-2023 as validation, and 2024 to evaluate the model

You can show the evaluation performance as a plot over time, or simply average the evaluation metric, or use Diebold-Mariano (DM) test.

Bias Variance Trade-off

Occam's Razor/Law of Parsimony

"Entities must not be multiplied beyond necessity." - Wikipedia

Classic wisdom: **simpler models are better.**

Bias-Variance Trade-off:

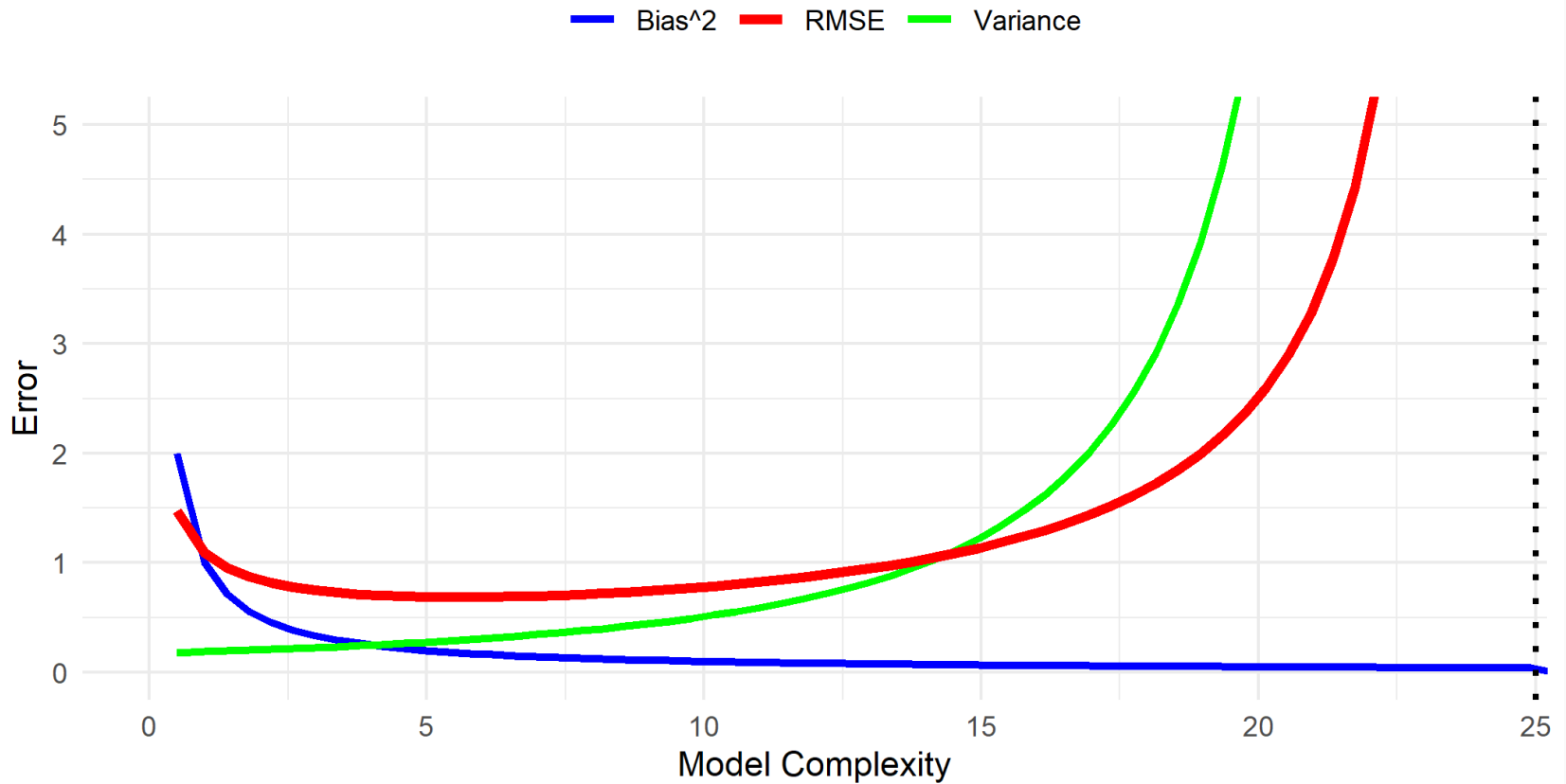
As model complexity increases, OOS performance: first improves -> then worsens

Forms a **U-shaped curve** for OOS RMSE.

Rule of thumb: Keep the model as simple as possible (“KISS”: Keep It Simple, Stupid).

Classical View

Classic Bias-Variance Tradeoff

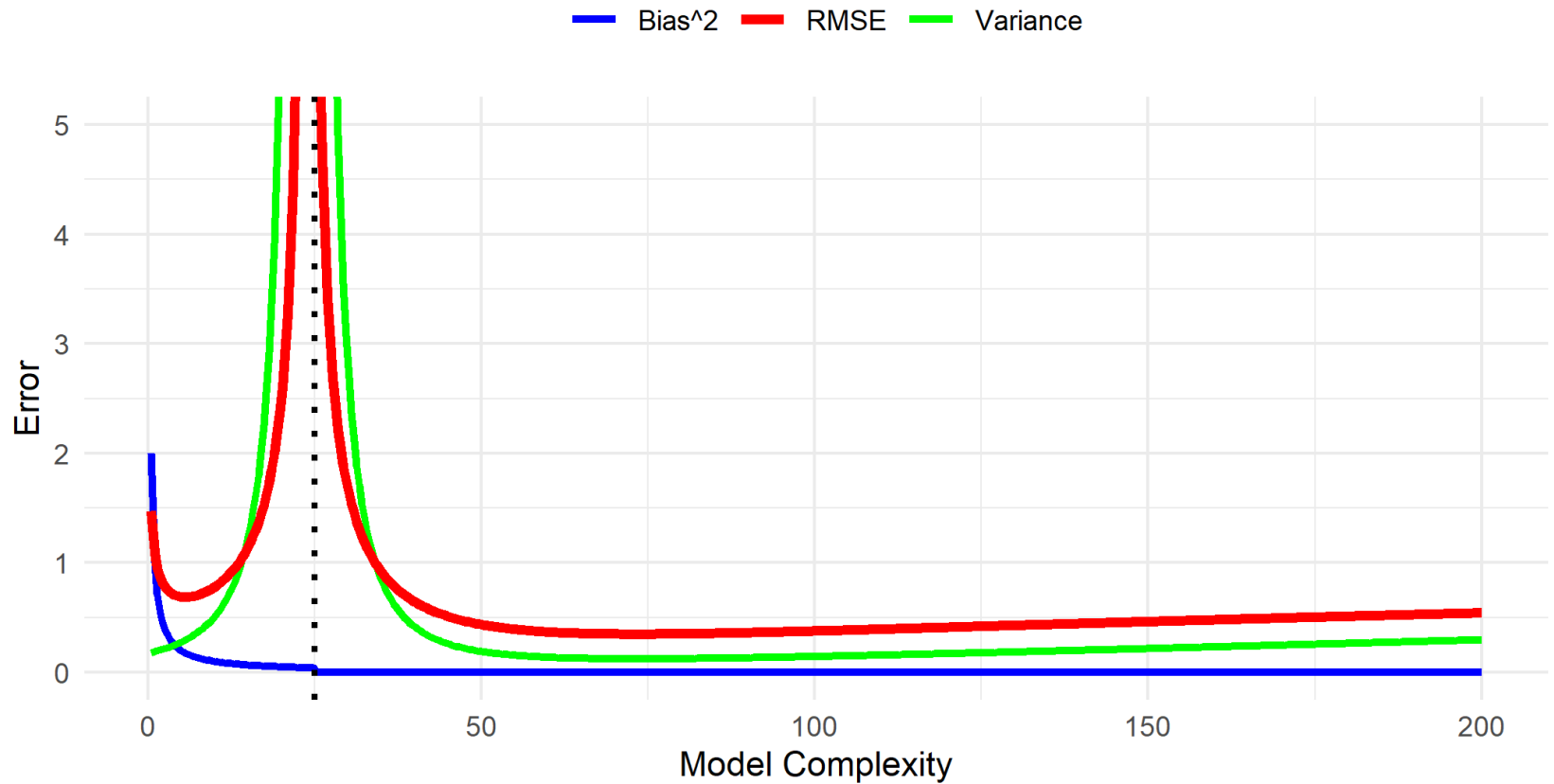


Modern Perspective: Double Descent

- Deep learning & LLMs challenge the simplicity rule.
- Complexity is **not necessarily bad**.
- If model complexity exceeds the interpolation point (parameters > observations):
 - OOS performance can improve **again**, forming a **double descent** curve.
- Modern compute power allows fitting **hypercomplex models** (e.g., deep neural networks) efficiently with strong predictive performance.

Double Descent

Double Descent



Next lecture(s): Complex Models: Trees and Neural Networks
